



CUỘC THI THIẾT KẾ VI MẠCH LẦN II – CHỦ ĐỀ: “THÀNH PHỐ XANH VÀ THÔNG MINH”

CUỘC THI THIẾT KẾ VI MẠCH: LỖI MÃ HÓA ASCON TỐI ƯU CHO THIẾT BỊ IOT

Các thành viên:

- | | |
|-----------------------|----------|
| • Nguyễn Hữu Học | 22520486 |
| • Nguyễn Đình Lê Mạnh | 22520848 |
| • Lương Trùng Dương | 22520301 |
| • Lê Trọng Điền | 22520246 |



NỘI DUNG BÁO CÁO

- 1 TỔNG QUAN ĐỀ TÀI
- 2 QUY TRÌNH THỰC HIỆN
- 3 KẾT QUẢ HIỆN THỰC
- 4 KẾT LUẬN



01

TỔNG QUAN ĐỀ TÀI



1. TỔNG QUAN ĐỀ TÀI

a) ASCON

- **Ascon** được phát triển vào năm 2014 bởi một nhóm các nhà mật mã từ Đại học Công nghệ Graz, Infineon Technologies, Intel Labs và Đại học Radboud dưới sự lãnh đạo của Christoph Dobraunig, Maria Eichlseder, Florian Mendel và Martin Schläffer. Thuật toán này lần đầu tiên được gửi đến cuộc thi CAESAR (Competition for Authenticated Encryption: Security, Applicability, and Robustness) vào năm 2014, nơi nó đã được công nhận là lựa chọn chính cho mật mã hóa được xác nhận là lightweight trong danh mục cuối cùng **được công bố vào tháng 2 năm 2019**. Sau các cuộc cạnh tranh và kiểm tra từ cộng đồng mật mã học, Ascon được **NIST chính thức chọn vào tháng 2 năm 2023** làm cơ sở cho quy trình chuẩn hóa mật mã nhẹ của mình, với tiêu chuẩn hoàn chỉnh được công bố trong NIST SP 800-232 vào tháng 8 năm 2025.



1. TỔNG QUAN ĐỀ TÀI

a) ASCON

- **Quyết định của NIST** phản ánh sự cần thiết ngày càng tăng đối với các giải pháp mật mã hiệu quả cho các thiết bị (IoT), cảm biến, và các hệ thống nhúng bị ràng buộc tài nguyên, nơi các tiêu chuẩn mật mã thông thường như AES-GCM có thể quá nặng nề. Ascon cung cấp một bộ mật mã cổ điển được thiết kế riêng để cân bằng hiệu suất, diện tích, tiêu thụ năng lượng và bảo mật, đặc biệt phù hợp với các trường hợp tài nguyên tính toán bị hạn chế nhưng bảo mật vẫn là ưu tiên tối cao.



1. TỔNG QUAN ĐỀ TÀI

b) Kiến trúc và Cấu trúc Sponge:

Cơ sở toán học và Cấu trúc Sponge:

- **Ascon** dựa trên bộ xây dựng lõi là **một hàm hoán vị 320 bit** được định nghĩa trên năm thanh ghi **64 bit** (x_0, x_1, x_2, x_3, x_4). Thiết kế này tuân theo mô hình **Substitution-Permutation Network (SPN)** với độ phức tạp thấp, cho phép các triển khai hiệu quả trên cả phần mềm và phần cứng.
- Khác với các cách tiếp cận truyền thống dựa trên Feistel hoặc khóa mở rộng, **Ascon** sử dụng **cấu trúc sponge/duplex** được khái niệm hóa lần đầu tiên bởi **Bertoni et al** để xây dựng các mật mã đa chức năng cổ điển.



1. TỔNG QUAN ĐỀ TÀI

b) Kiến trúc và Cấu trúc Sponge:

Hàm Hoán vị Ascon:

- **Hàm hoán vị p** Được xây dựng dựa trên **mạng thay thế - hoán vị (SPN)**, thực hiện **lặp lại 3 bước xử lý tuần tự** trong mỗi vòng:
 1. **Lớp bổ sung hằng số (pC):** **XOR một hằng số 64-bit** đặc trưng cho từng vòng (c_i) từ trạng thái thứ ba (x_2). Hằng số này được thiết kế để ngăn ngừa các tấn công sử dụng cấu trúc lặp.
 2. **Lớp thay thế (pS):** Triển khai **64 bộ S-box 5-bit hoạt động song song** theo kiến trúc **bit-sliced**. S-box được xây dựng từ các hàm Boolean 5-bit có **bậc đại số thấp (bậc 2)**, tạo điều kiện thuận lợi để áp dụng các kỹ thuật **Masking** nhằm chống lại tấn công **kênh kề (Side-channel)**.
 3. **Lớp khuếch tán tuyến tính (pL):** Áp dụng các **phép xoay và XOR** để khuếch tán sự thay đổi trong các từ trạng thái. Các phép xoay được chọn để đảm bảo tính khuếch tán nhanh trong mạng SPN.



1. TỔNG QUAN ĐỀ TÀI

b) Kiến trúc và Cấu trúc Sponge:

Hàm Hoán vị Ascon:

- Công thức cho lớp khuếch tán tuyến tính (Linear_Diffusion):
 - $\Sigma_0(x_0) = x_0 \oplus (x_0 \ggg 19) \oplus (x_0 \ggg 28)$
 - $\Sigma_1(x_1) = x_1 \oplus (x_1 \ggg 61) \oplus (x_1 \ggg 39)$
 - $\Sigma_2(x_2) = x_2 \oplus (x_2 \ggg 1) \oplus (x_2 \ggg 6)$
 - $\Sigma_3(x_3) = x_3 \oplus (x_3 \ggg 10) \oplus (x_3 \ggg 17)$
 - $\Sigma_4(x_4) = x_4 \oplus (x_4 \ggg 7) \oplus (x_4 \ggg 41)$



1. TỔNG QUAN ĐỀ TÀI

b) Kiến trúc và Cấu trúc Sponge:

- Công thức hàm thay thế (Substitution):

$$s_0 = (s_4 \cdot s_1) \oplus s_3 \oplus (s_2 \cdot s_1) \oplus s_2 \oplus (s_1 \cdot s_0) \oplus s_1 \oplus s_0$$

$$s_1 = s_4 \oplus (s_2 \cdot s_3) \oplus s_3 \oplus (s_3 \cdot s_1) \oplus s_2 \oplus (s_1 \cdot s_2) \oplus s_1 \oplus s_0$$

$$s_2 = (s_4 \cdot s_3) \oplus s_4 \oplus s_2 \oplus s_1 \oplus 1$$

$$s_3 = (s_4 \cdot s_0) \oplus (s_3 \cdot s_0) \oplus s_4 \oplus s_3 \oplus s_2 \oplus s_1 \oplus s_0$$

$$s_4 = (s_4 \cdot s_1) \oplus s_4 \oplus s_3 \oplus (s_1 \cdot s_0) \oplus s_1$$



1. TỔNG QUAN ĐỀ TÀI

b) Kiến trúc và Cấu trúc Sponge:

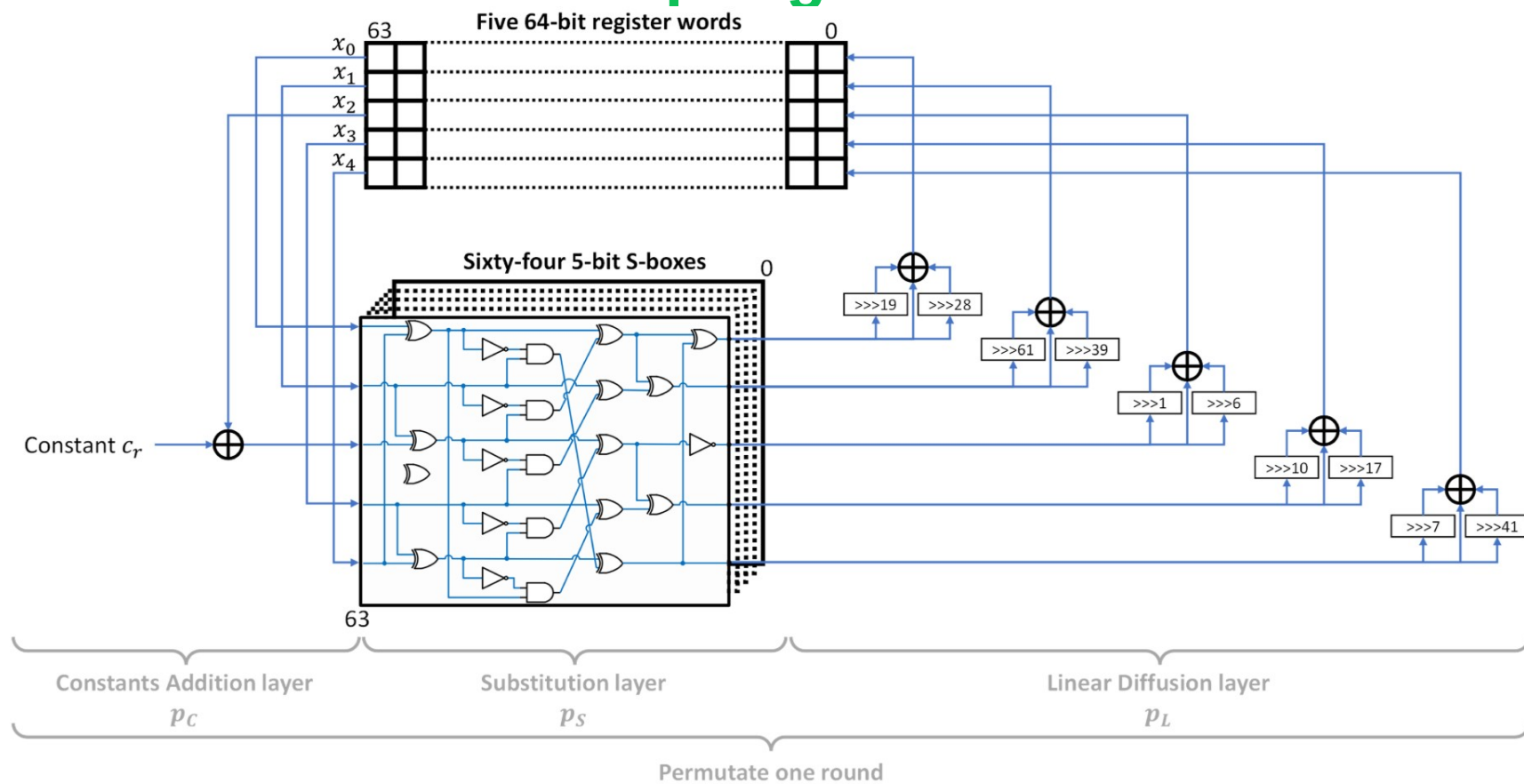
Bảng hằng số Cr :

			Constant
0			0000000000000000f0
1			0000000000000000e1
2			0000000000000000d2
3			0000000000000000c3
4	0		0000000000000000b4
5	1		0000000000000000a5
6	2	0	000000000000000096
7	3	1	000000000000000087
8	4	2	000000000000000078
9	5	3	000000000000000069
10	6	4	00000000000000005a
11	7	5	00000000000000004b



1. TỔNG QUAN ĐỀ TÀI

b) Kiến trúc và Cấu trúc Sponge:

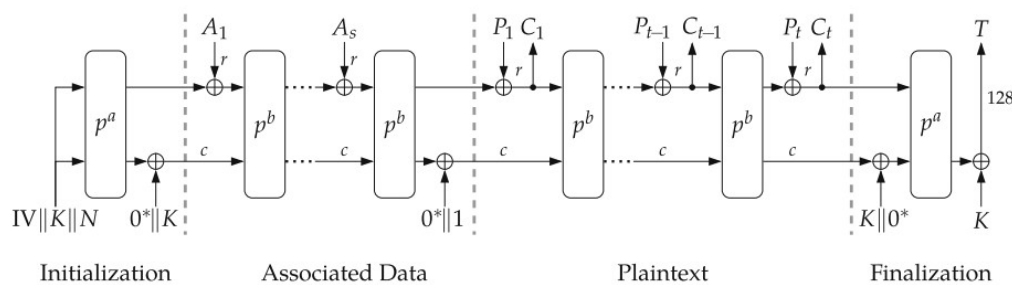




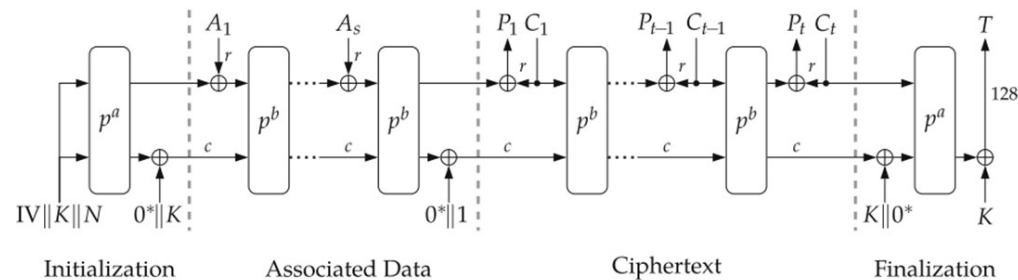
1. TỔNG QUAN ĐỀ TÀI

b) Kiến trúc và Cấu trúc Sponge:

- **Cấu trúc Sponge Duplex** (lấy cảm hứng MonkeyDuplex, Bertoni et al.).
- Trạng thái nội bộ: 320-bit (5 từ 64-bit: x_0 - x_4), chia rate (r) và capacity (c).
- Quy trình: **Initialization** (khóa, nonce, IV), **Associated Data Processing**, **Plaintext Processing**, **Finalization** (tạo tag).



(a) Encryption $\mathcal{E}_{k,r,a,b}$



(b) Decryption $\mathcal{D}_{k,r,a,b}$



02

QUY TRÌNH THỰC HIỆN



2. QUY TRÌNH THỰC HIỆN

a) NGHIÊN CỨU VÀ LÊN Ý TƯỞNG CHI TIẾT

Phân tích yêu cầu: Xác định rõ đối tượng thiết bị IoT (cảm biến siêu nhỏ, bộ điều khiển, gateway), yêu cầu về thông lượng, độ trễ, diện tích chip và ngân sách năng lượng.

Nghiên cứu thuật toán ASCON: Tài liệu chính thức của NIST về ASCON-AEAD128 và ASCON-AEAD128a. Hiểu sâu về cấu trúc sponge, phép hoán vị Ascon-p (S-box, P-box, C-box), các giai đoạn Initialization, AD processing, Plaintext/Ciphertext processing, và Finalization.



2. QUY TRÌNH THỰC HIỆN

a) NGHIÊN CỨU VÀ LÊN Ý TƯỞNG CHI TIẾT

Tham khảo triển khai hiện có: Tìm kiếm và phân tích các mã nguồn tham chiếu phần mềm và các nghiên cứu về triển khai phần cứng của ASCON để học hỏi từ các giải pháp đã có.

Lựa chọn kiến trúc phần cứng: Đây là quyết định then chốt. Dựa trên yêu cầu, lựa chọn giữa:

- **Kiến trúc vòng lặp (Iterative):** Tối ưu cho diện tích nhỏ và năng lượng thấp, phù hợp với hầu hết các thiết bị IoT. Đây là lựa chọn chính cho dự án này.
- **Kiến trúc đường ống (Pipelined):** Nếu cần thông lượng cao hơn và có thể chấp nhận diện tích lớn hơn. Không phù hợp với tiêu chí đề tài.



2. QUY TRÌNH THỰC HIỆN

b) THIẾT KẾ RTL (REGISTER-TRANSFER LEVEL)

Xây dựng cấu trúc module: Chia lõi ASCON thành các module nhỏ, dễ quản lý và kiểm tra:

- **ascon_core:** Module điều khiển chính, quản lý giao diện và luồng dữ liệu. Đồng thời xử lý toàn bộ máy trạng thái, tính toán, padding, dữ liệu.
- **ascon_permutation:** Module triển khai phép hoán vị Ascon-p 320-bit, bao gồm các phép toán XOR, AND, NOT, rotate và các S-box.
- **ascon_top:** Gồm các logic nạp giá trị cho CORE, với đầu vào 32bit và đầu ra theo chế độ hoạt động và thể loại ascon đang dùng (variant)
- **ascon_wb:** Giao tiếp với bus wishbone của RiscV steel, cùng với controller điều tiết nạp dữ liệu cho khối ascon_top



2. QUY TRÌNH THỰC HIỆN

b) THIẾT KẾ RTL (REGISTER-TRANSFER LEVEL)

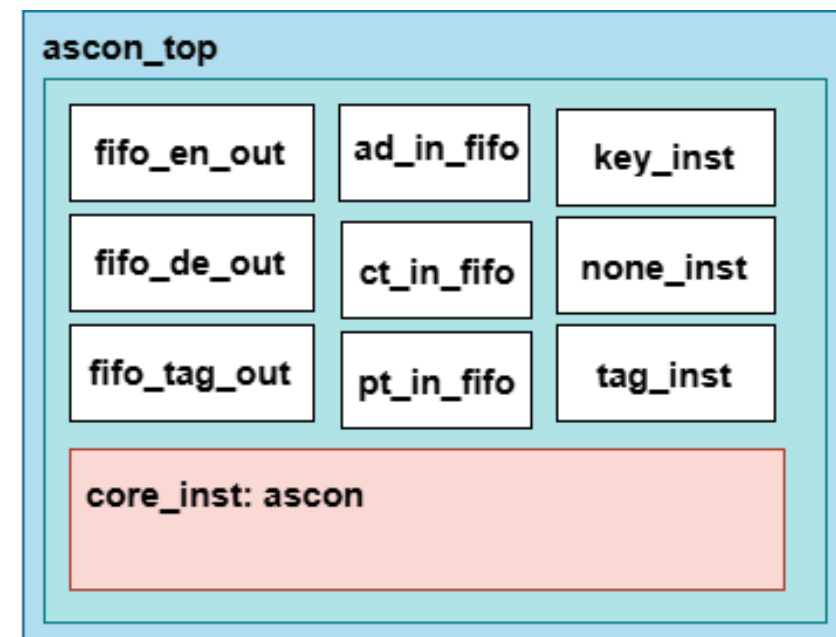
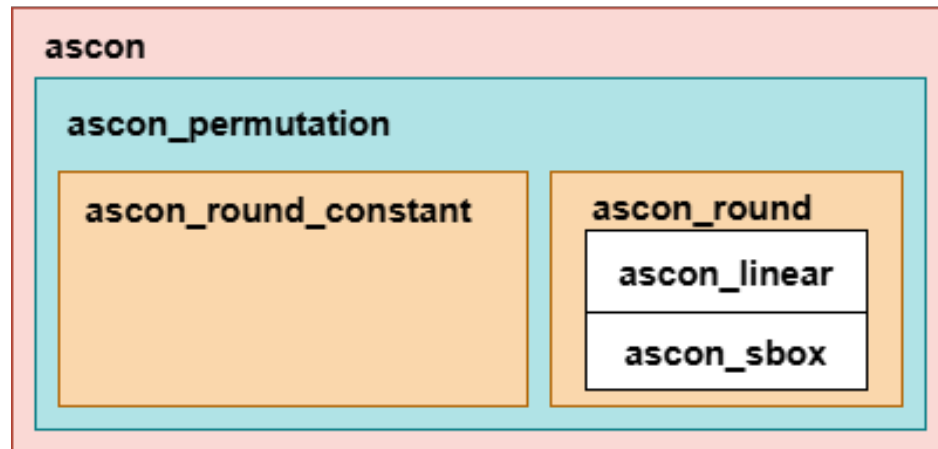
Xây dựng cấu trúc module: Chia lõi ASCON thành các module nhỏ, dễ quản lý và kiểm tra:

- **assembler:** Ghép các giá trị 32bit thành giá trị có độ dài phù hợp với ascon core.
- **splitter:** Chia giá trị đầu ra thành các pack dữ liệu 32bit để có thể truyền qua wishbone
- **FIFO:** Kết hợp với assembler và splitter để đưa dữ liệu vào khối ascon xử lý



2. QUY TRÌNH THỰC HIỆN

c) TỔNG QUAN THIẾT KẾ:

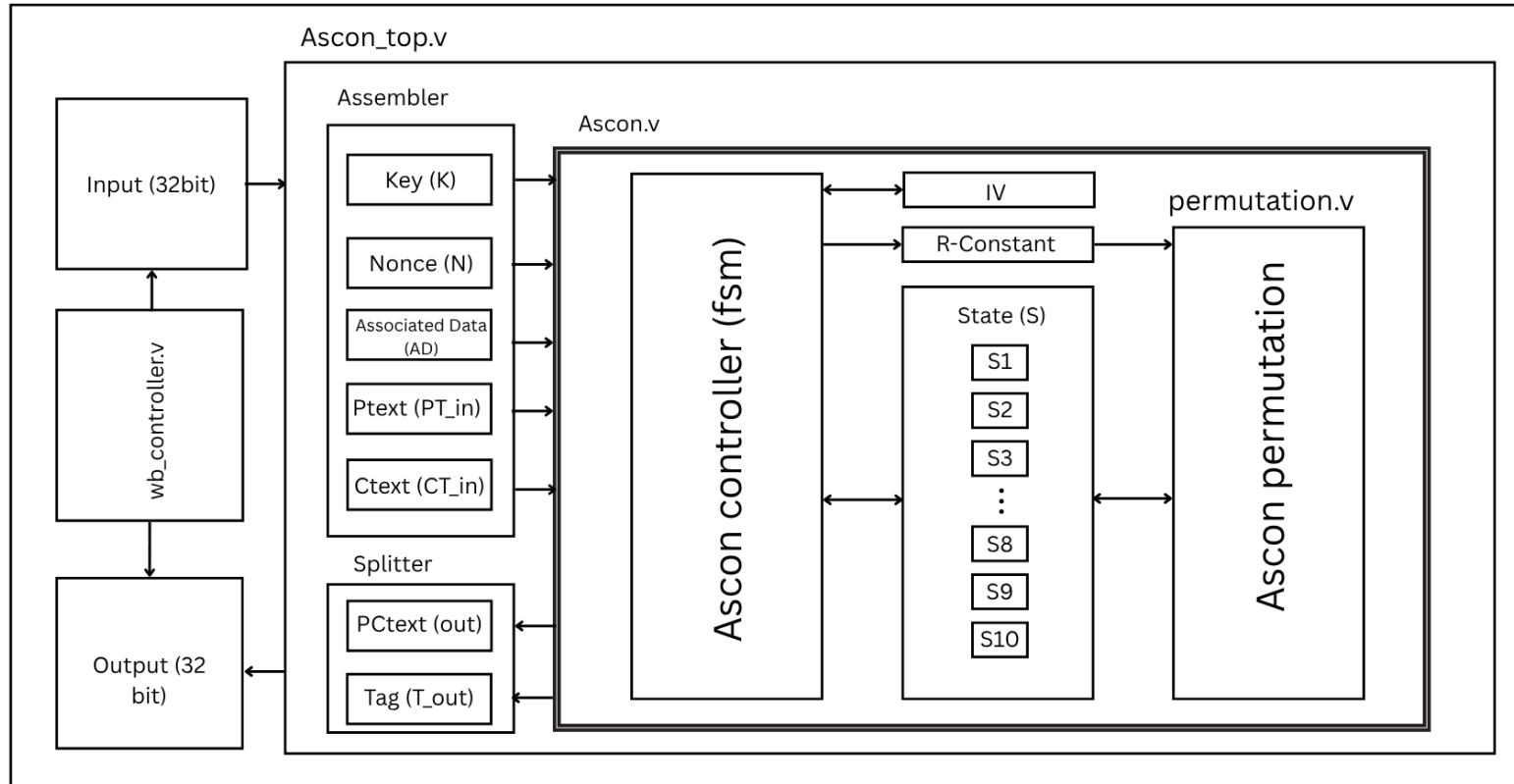




2. QUY TRÌNH THỰC HIỆN

c) TỔNG QUAN THIẾT KẾ:

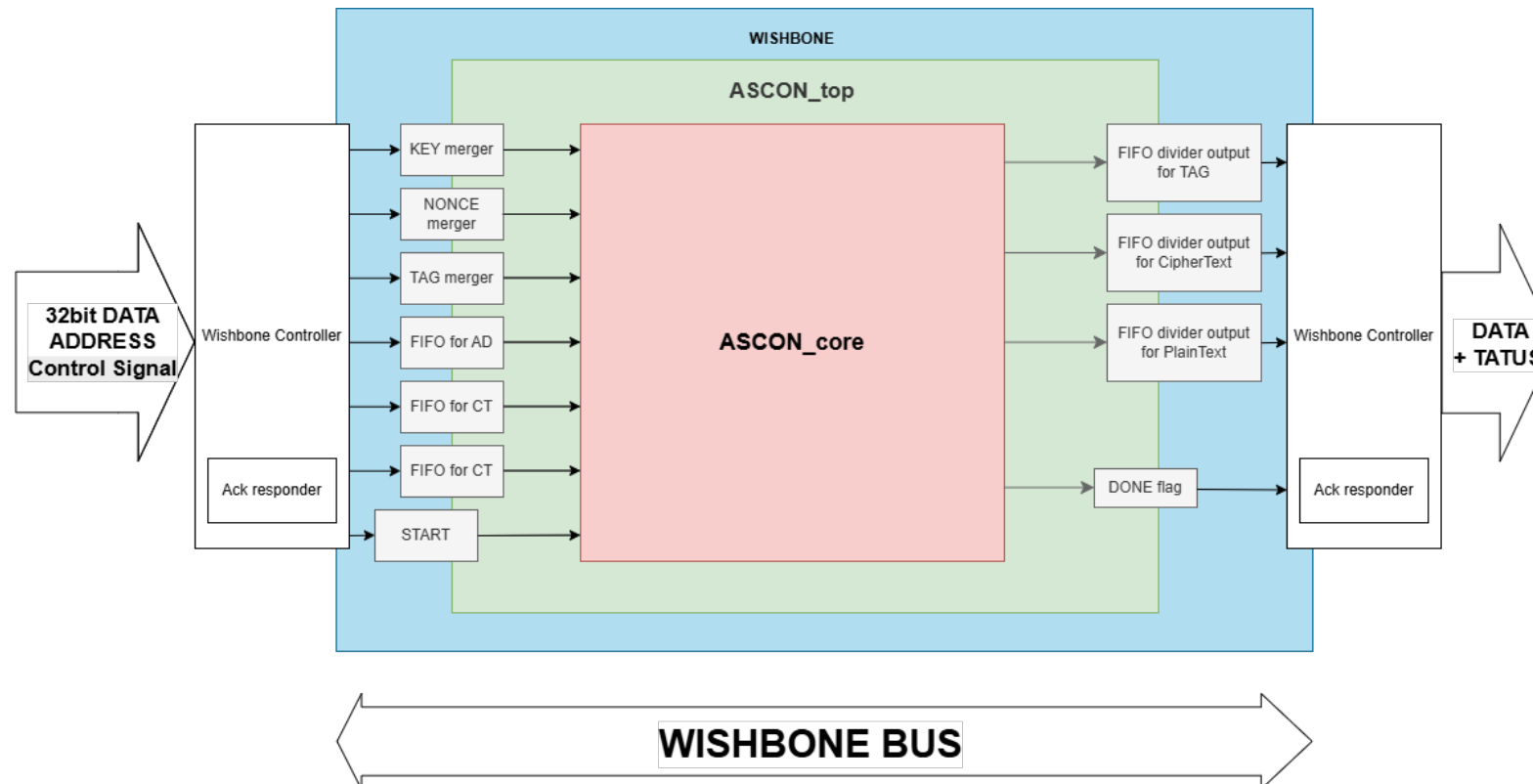
wb_ascon.v





2. QUY TRÌNH THỰC HIỆN

c) TỔNG QUAN THIẾT KẾ:

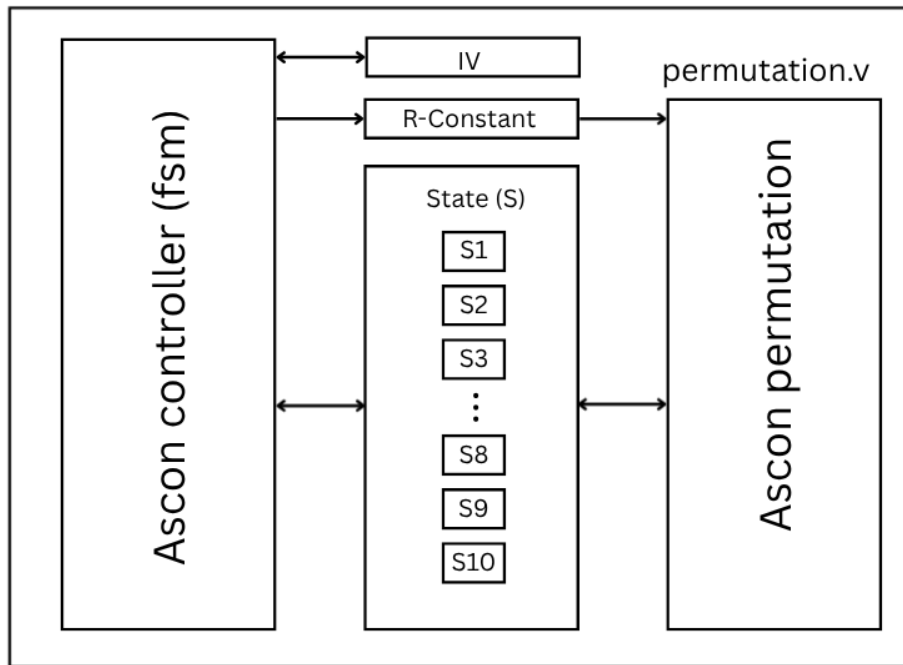




2. QUY TRÌNH THỰC HIỆN

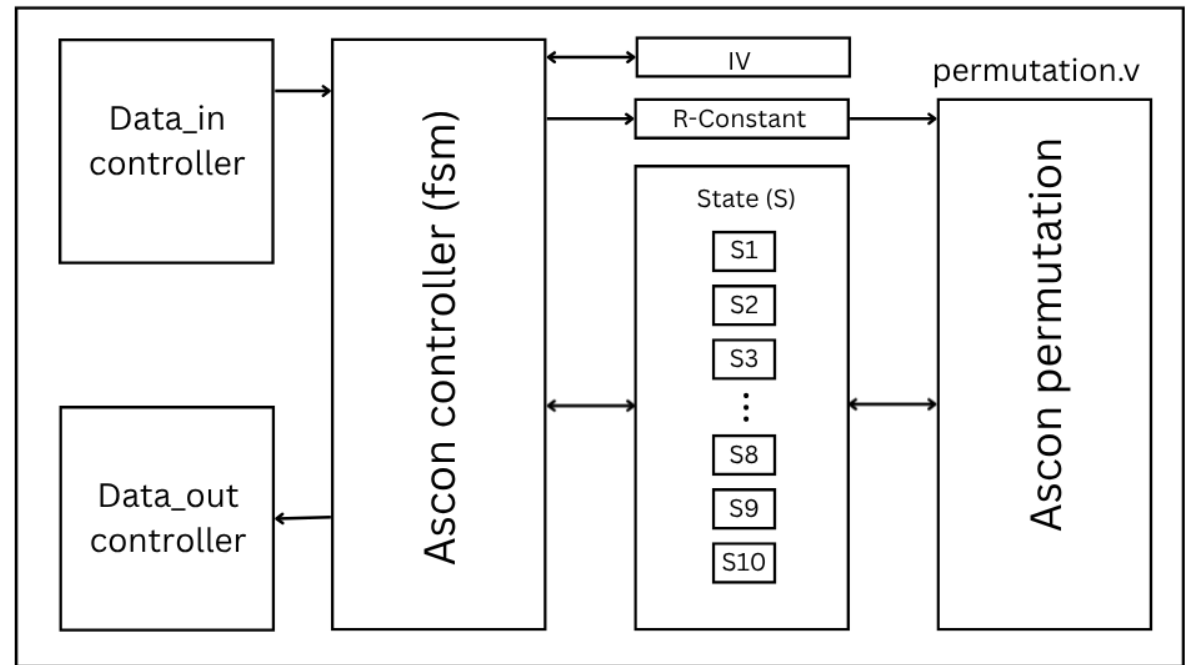
d) CHỈNH SỬA VÀ CẢI TIẾN THIẾT KẾ :

Ascon.v



Ascon.v (core cũ)

Ascon.v



Ascon.v (core mới)



2. QUY TRÌNH THỰC HIỆN

d) CHỈNH SỬA VÀ CẢI TIẾN THIẾT KẾ :

Đặc điểm	ascon.v (Original)	ascon_optimized.v (Optimized)	Lợi ích của bản Optimized
Bus Dữ liệu Input	4 Bus riêng biệt: ad_din (128b), pt_din (128b), ct_din (128b), tag_din (128b) Total Data Pins ≈ 512 bits	1 Bus thống nhất: data_in (128b) Total Data Pins = 128 bits	Giảm 75% số lượng dây nối dữ liệu input. Tiết kiệm tài nguyên routing cực lớn.
Tín hiệu Điều khiển	Nhiều cặp Req/Valid riêng lẻ (ad_req, pt_req...)	1 cặp Handshake chung (data_valid, data_ready) + data_type	Đơn giản hóa logic điều khiển ở lớp trên (Top-level Controller).
Bus Dữ liệu Output	Tách riêng e_ciphertext (Mã hóa) và d_received_text (Giải mã).	Gom chung thành out_data cho cả Encrypt/Decrypt.	Giảm thêm 128-bit dây nối ở output.
Cơ chế xác định dữ liệu	Dựa vào việc dữ liệu nằm trên dây nào ("Positional").	Dựa vào mã data_type đi kèm ("Tagged").	Linh hoạt hơn, giống giao thức mạng/streaming.
Hardware Overhead	Cần nhiều Flip-Flop/Buffer ở I/O nếu synthesis tool không tối ưu tốt.	Cần thêm một chút logic MUX (bộ dồn kênh) để phân loại dữ liệu bên trong.	Logic MUX tốn rất ít tài nguyên so với chi phí của hàng trăm chân I/O.



03

KẾT QUẢ HIỆN THỰC



3. KẾT QUẢ HIỆN THỰC

Thiết kế (Nguồn)	Thiết bị (Device)	Tài nguyên (LUTs)	Throughput (Mbps)	Hiệu quả (Mbps/LUT)	Ghi Chú
This work	Virtex-7	1,968	1,116 (128a) 426 (128)	0.83	Iterative (1 round/clock)
Alharbi et al. 1	Virtex-7	1,632	914 (128a) 400 (128)	0.56	Iterative (1 round/clock)
Alharbi et al. 2	Kintex-7	1,497	914 (128a)	0.61	Tối ưu diện tích tốt nhất
Khan et al. 3	Virtex-7	2,708	721.5	0.26	Recursive Architecture
Tran et al. 4	Virtex-7	6,536	13,312	02.03	Pipelined/Unrolled (Hiệu năng cao)
Kurian & Chen 5	Artix-7	5,335	~800	0.15	2 rounds/clock

BẢNG TỔNG HỢP SO SÁNH



3. KẾT QUẢ HIỆN THỰC

Thông số	Thiết kế (This work)	Alharbi et al	Chênh lệch (%)
Platform	Virtex-7	Virtex-7	-
Tần số (Frequency)	200 MHz	335 MHz	-40.3%
Tài nguyên (LUTs)	1,968	1,632	+20.6%
Thanh ghi (FFs)	1,496	912	+64.0%
Công suất (Power)	377 mW	239 mW	+57.7%
Throughput (Ascon-128)	426 Mbps	400 Mbps	+6.5%
Throughput (Ascon-128a)	1,116 Mbps	914 Mbps	+22.1%
Efficiency (Ascon-128)	0.21 Mbps/LUT	0.25 Mbps/LUT	-
Efficiency (Ascon-128a)	0.59 Mbps/LUT	0.56 Mbps/LUT	-

Bảng so sánh Hiệu Năng Phần cứng Ascon trên Virtex-7



3. KẾT QUẢ HIỆN THỰC

```
test 3
  Start 3: AsconBenchmarks

3: Test command: /home/huuhoc/riscv_workspace/install/bin/spike "--isa=riscv32im_zicsr_zifencei_zicntr_zbkb" "/home/huuhoc/riscv_workspace/install/riscv32-unknown-elf/bin/pk" "/home/huuhoc/riscv_workspace/lightweight_aead/build/ascon_bench/ascon_bench"
3: Test timeout computed to be: 10000000
3: Running tests...
3:
3: ascon_128/encrypt: 16/954.0, 32/190.6, 64/130.3, 128/100.1, 256/85.1, 512/77.5, 1024/73.8, 2048/71.9, 4096/71.8, 8192/70.9, 16384/70.6,
3: ascon_128/decrypt: 16/313.9, 32/190.2, 64/128.3, 128/97.4, 256/82.0, 512/74.2, 1024/70.4, 2048/68.4, 4096/67.5, 8192/67.0, 16384/66.7,
3: ascon_128a/encrypt: 16/267.8, 32/157.8, 64/102.8, 128/75.3, 256/61.5, 512/54.6, 1024/51.2, 2048/49.5, 4096/48.6, 8192/48.2, 16384/48.0,
3: ascon_128a/decrypt: 16/270.4, 32/157.3, 64/100.7, 128/72.4, 256/58.3, 512/51.2, 1024/47.7, 2048/45.9, 4096/45.0, 8192/44.6, 16384/44.3,
3:
3:
3: 0 tests passed, 0 tests failed
3/5 Test #3: AsconBenchmarks ..... Passed    0.07 sec
```

**Benchmark sử dụng spike simulation chạy bare-metal
riscv32im_zbkb**



3. KẾT QUẢ HIỆN THỰC

```
=====
ASCON-128a BENCHMARK REPORT (200 MHz)
Metric: Cycles Per Byte (CPB) = (Total Cycles) / (Payload Bytes)
Includes: Initialization + Processing + Finalization
=====

Starting Benchmark for 16 bytes
Im here 11 - DONE RECEIVED
Bytes: 16 | Cycles: 35 | CPB: 2.19
Starting Benchmark for 32 bytes
Im here 11 - DONE RECEIVED
Bytes: 32 | Cycles: 46 | CPB: 1.44
Starting Benchmark for 64 bytes
Im here 11 - DONE RECEIVED
Bytes: 64 | Cycles: 68 | CPB: 1.06
Starting Benchmark for 80 bytes
Im here 11 - DONE RECEIVED
Bytes: 80 | Cycles: 79 | CPB: 0.99
Starting Benchmark for 128 bytes
Im here 11 - DONE RECEIVED
Bytes: 128 | Cycles: 112 | CPB: 0.88
Starting Benchmark for 256 bytes
Im here 11 - DONE RECEIVED
Bytes: 256 | Cycles: 200 | CPB: 0.78
Starting Benchmark for 512 bytes
Im here 11 - DONE RECEIVED
Bytes: 512 | Cycles: 376 | CPB: 0.73
Starting Benchmark for 1024 bytes
Im here 11 - DONE RECEIVED
Bytes: 1024 | Cycles: 728 | CPB: 0.71
Starting Benchmark for 8192 bytes
Im here 11 - DONE RECEIVED
Bytes: 8192 | Cycles: 5656 | CPB: 0.69
=====

BENCHMARK COMPLETE
=====
```

Benchmark
simulation từ
testbench

```
=====
ASCON-128 BENCHMARK REPORT (200 MHz)
Metric: Cycles Per Byte (CPB) = (Total Cycles) / (Payload Bytes)
Includes: Initialization + Processing + Finalization
=====

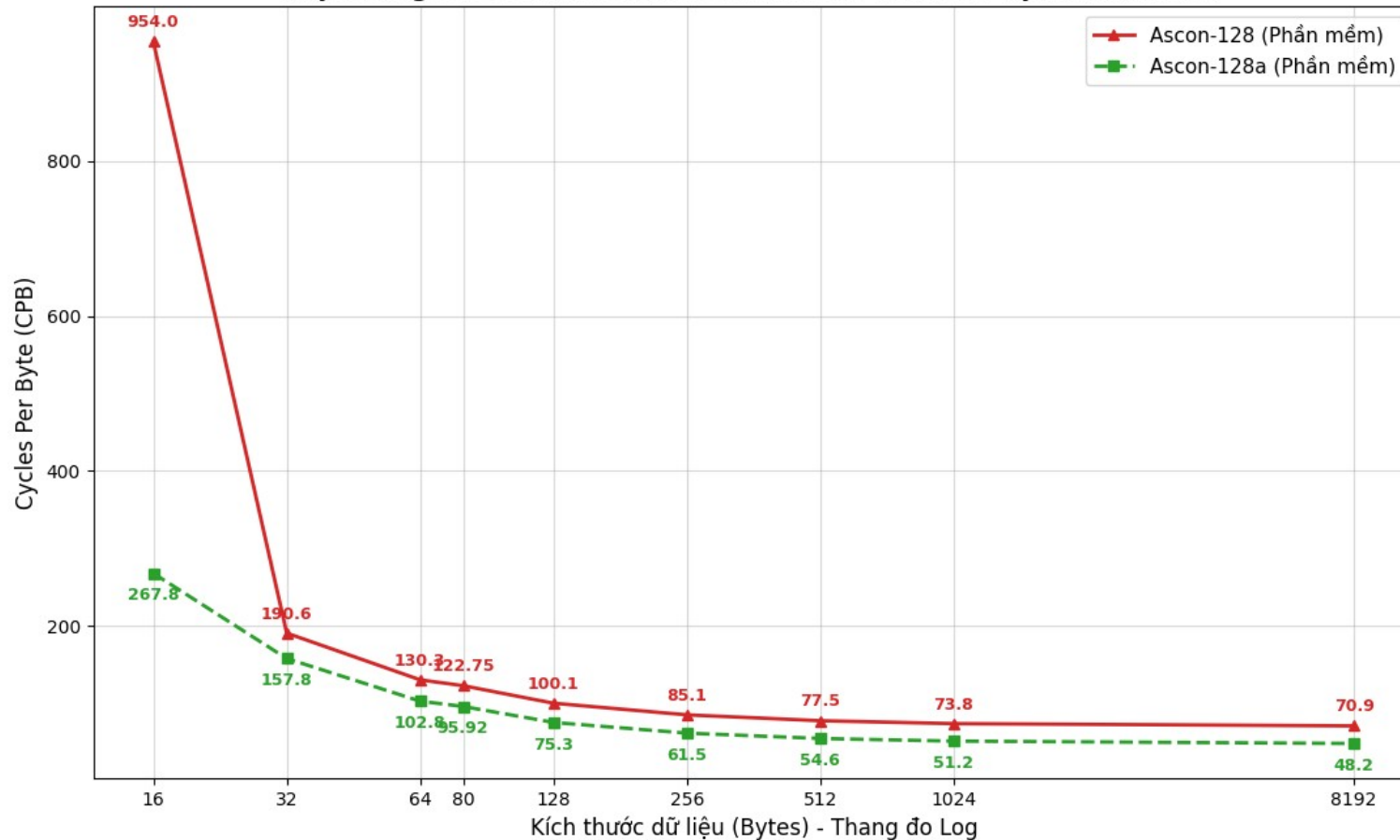
Starting Benchmark for 16 bytes
Im here 11 - DONE RECEIVED
Bytes: 16 | Cycles: 35 | CPB: 2.19
Starting Benchmark for 32 bytes
Im here 11 - DONE RECEIVED
Bytes: 32 | Cycles: 44 | CPB: 1.38
Starting Benchmark for 64 bytes
Im here 11 - DONE RECEIVED
Bytes: 64 | Cycles: 62 | CPB: 0.97
Starting Benchmark for 80 bytes
Im here 11 - DONE RECEIVED
Bytes: 80 | Cycles: 71 | CPB: 0.89
Starting Benchmark for 128 bytes
Im here 11 - DONE RECEIVED
Bytes: 128 | Cycles: 98 | CPB: 0.77
Starting Benchmark for 256 bytes
Im here 11 - DONE RECEIVED
Bytes: 256 | Cycles: 170 | CPB: 0.66
Starting Benchmark for 512 bytes
Im here 11 - DONE RECEIVED
Bytes: 512 | Cycles: 314 | CPB: 0.61
Starting Benchmark for 1024 bytes
Im here 11 - DONE RECEIVED
Bytes: 1024 | Cycles: 602 | CPB: 0.59
Starting Benchmark for 8192 bytes
Im here 11 - DONE RECEIVED
Bytes: 8192 | Cycles: 4634 | CPB: 0.57
=====

BENCHMARK COMPLETE
=====
```



3. KẾT QUẢ HIỆN THỰC

Hiệu năng CPB của Ascon-128 và Ascon-128a (Dữ liệu Phần mềm)

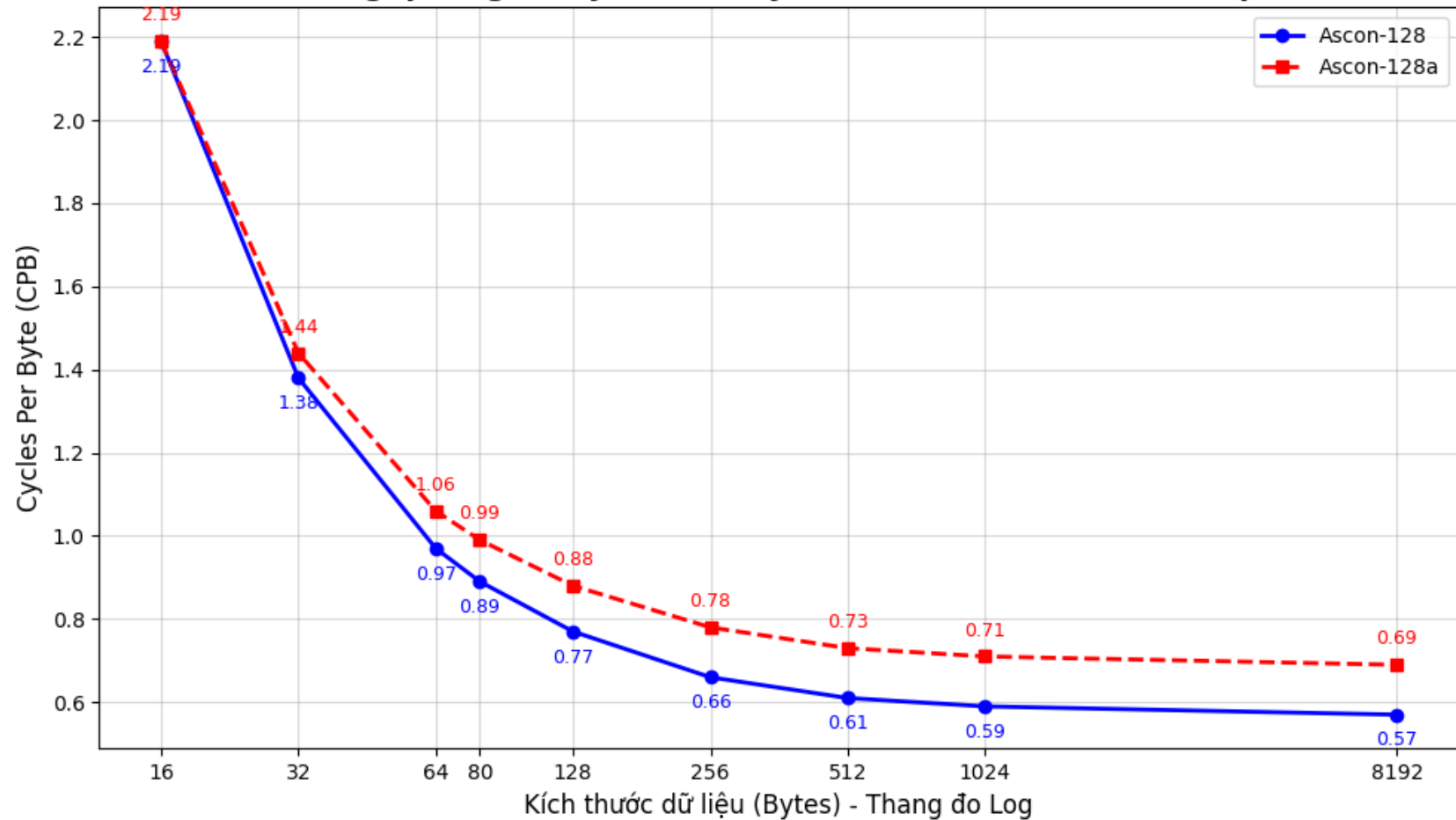


Thông số cbp thực hiện
benchmark bare-metal
trên riscv32im_zbkb
(extention for crypton)



3. KẾT QUẢ HIỆN THỰC

Tương quan giữa Cycles Per Byte (CPB) và Kích thước dữ liệu





3. KẾT QUẢ HIỆN THỰC

- Benchmark cho thấy các test encryption/decryption trên nền tảng RISC-V rv32im_zbkb đạt tốc độ xử lý lần lượt là **70.6–954 chu kỳ/byte** tùy theo kích thước dữ liệu
- Đồng thời cho thấy cải thiện một phần hiệu năng so với các bài báo cùng sử dụng một nền tảng FPGA (Virtex-7).
- Nhằm tối ưu, cũng như sử dụng - cài đặt trên các thiết bị IoT nhỏ gọn, bắt buộc phải tối ưu Logic Element, FF, điện năng tiêu thụ.



3. KẾT QUẢ HIỆN THỰC

This is plaintext from Ascon, if you see this, it worked as intended

Character encoding

ASCII

Output delimiter string (optional)

None

= Convert

× Reset

↕ Swap

```
5468697320697320706C61696E746578742066726F6D204173636F6E2  
C20696620796F752073656520746869732C20697420776F726B656420  
617320696E74656E646564
```



3. KẾT QUẢ HIỆN THỰC

Encrypt

key:

nonce:

plaintext:

associated data:

ciphertext: 72e9babcf32889697678d9b1ac9be9c34ae2fde35361bc00bb2b93724ff8478d7cc71072808c3c7dac047c749181bb01dfd23740706a0f58ab50438d32c66f8ef43b44514b2ff09e
tag: 9caeb5f8dce88b55ee1682f1eb849354

=====

Decrypt

key:



3. KẾT QUẢ HIỆN THỰC

Kịch bản test: Dùng trang web playascon để lấy dữ liệu mẫu để test, sau đó nạp dữ liệu mẫu như đã test. Nếu encrypt, không cần nạp tag và ciphertext (CT). Nếu decrypt, không cần nạp plaintext (PT). Sau khi đã nạp dữ liệu đầy đủ, dùng acc_start để bắt đầu Sử dụng vòng lặp while để liên tục check status bằng acc_check_stat, nếu trả về 1 sẽ bắt đầu đọc dữ liệu bằng acc_read_en hoặc acc_read_tag nếu encrypt, nếu decrypt sẽ dùng mỗi acc_read_tag

Các lệnh điều khiển ascon:

```
//mode <= wb_dat_i[0:0]; 1 DE / 0 EN
//crypt_variant <= wb_dat_i[2:1];
00 01 10 void acc_setup (uint32_t data);
void acc_key_in (uint32_t data);
void acc_nonce_in (uint32_t data);
void acc_tag_in (uint32_t data);
void acc_ad_in (uint32_t data);
void acc_pt_in (uint32_t data);
void acc_ct_in (uint32_t data);
void acc_start (uint32_t data);
uint32_t acc_check_stat (void);
uint32_t acc_read_en (void);
uint32_t acc_read_de (void);
uint32_t acc_read_tag (void);
```



04

KẾT LUẬN



4. KẾT LUẬN

Thiết kế lỗi mã hóa ASCON không chỉ là một bài toán kỹ thuật mà còn là một bước tiến quan trọng trong việc **nâng cao bảo mật cho hệ sinh thái IoT**.

Bằng cách cung cấp một giải pháp mã hóa hiệu quả, nhỏ gọn và an toàn, dự án này sẽ góp phần xây dựng một tương lai nơi dữ liệu IoT được truyền tải và xử lý một cách đáng tin cậy, ngay cả trong các môi trường tài nguyên hạn chế nhất.



4. KẾT LUẬN

Tài liệu tham khảo

1. <https://ieeexplore.ieee.org/iel8/6287639/10380310/10701064.pdf>
2. <https://arxiv.org/pdf/2304.06222.pdf>
3. <https://csrc.nist.gov/csrc/media/Events/2023/lightweight-cryptography-workshop-2023/documents/accepted-papers/07-hardware-implementation-of-ascon.pdf>
4. https://iris.polito.it/retrieve/8066240e-42d6-4c56-a5ad-cc58af68638f/CHIPS___Ascon_survey.pdf
5. <https://csrc.nist.gov/csrc/media/Events/2023/lightweight-cryptography-workshop-2023/documents/accepted-papers/14-fpga-implementations-of-message-authentication-codes.pdf>
6. <https://mediatum.ub.tum.de/doc/1591312/1591312.pdf>
7. <https://docs.amd.com/api/khub/documents/2LByHkO~nSZXcei2D55fTg/content>
8. <https://idus.us.es/bitstreams/cb98c78c-9e23-4e36-ab71-2dabc914e18d/download>
9. https://iris.polito.it/retrieve/handle/11583/2999171/8066240e-42d6-4c56-a5ad-cc58af68638f/CHIPS___Ascon_survey.pdf
10. <https://csrc.nist.gov/CSRC/media/Events/lightweight-cryptography-workshop-2020/documents/papers/fpga-benchmarking-lwc2020.pdf>



4. KẾT LUẬN

Tài liệu tham khảo

11. <https://webthesis.biblio.polito.it/33812/1/tesi.pdf>
12. <https://dergipark.org.tr/en/download/article-file/4355237>
13. <https://digitalcommons.usf.edu/cgi/viewcontent.cgi?article=1126&context=mca>
14. <https://www.scribd.com/document/852902903/Evaluating-Ascon-Hardware-on-7-Series-FPGA-Devices>
15. <https://uh-ir.tdl.org/server/api/core/bitstreams/5b87bf97-6abd-4760-aa5c-63deb5c51ecb/content>
16. https://pure.qub.ac.uk/files/614755321/10207_2024_Article_904.pdf
17. [https://www.techrxiv.org/users/915346/articles/1290588/master/file/data/KECCAK_Permuted_ASCON___A_Novel_Hardware_for_Cryptography%20\(1\)/KECCAK_Permuted_ASCON___A_Novel_Hardware_for_Cryptography%20\(1\).pdf?inline=true](https://www.techrxiv.org/users/915346/articles/1290588/master/file/data/KECCAK_Permuted_ASCON___A_Novel_Hardware_for_Cryptography%20(1)/KECCAK_Permuted_ASCON___A_Novel_Hardware_for_Cryptography%20(1).pdf?inline=true)